



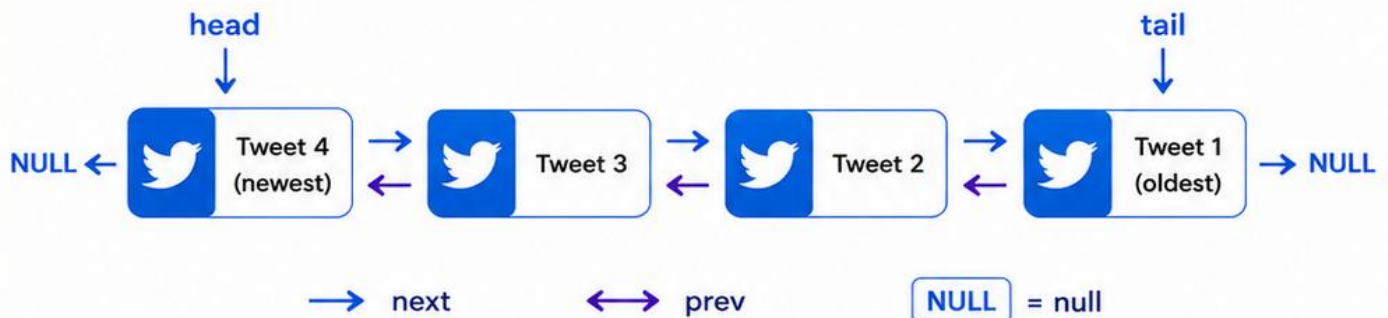
TWITTER TIMELINE



— ● ● ● —

— BASED ON LINKED LIST —

*A Project Report on Implementation of
Singly & Doubly Linked List in C++*



Efficient. Dynamic. Connected.
Just like your timeline.



Submitted By
POOJA KHANDELWAL



Department
BCA



Semester
2ND



Session
2025 - 2026

C++ Node Definition (Doubly Linked List)

```
class Node {  
public:  
    int id;  
    string userName;  
    string tweet;  
    Node* prev; // pointer to previous node  
    Node* next; // pointer to next node  
  
    // Constructor  
    Node(int _id, string _user, string _tweet) {  
        id = _id;  
        userName = _user;  
        tweet = _tweet;  
        prev = nullptr;  
        next = nullptr;  
    }  
};
```



DEZYNE ÉCOLE COLLEGE
Today A Reader, Tomorrow A Leader

www.dezyneecole.com | 2025-2026



Dezyne École College
www.dezyneecole.com

Title of Project

Twitter Timeline Based on Link List

Subject Name

Data Structure and C++

Submitted By

Pooja Khandelwal

Department – BCA

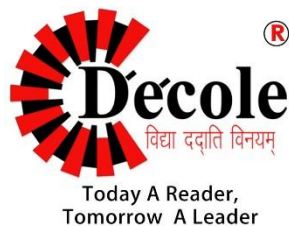
Semester – 2nd

Session – 2025 - 2026

Project Report on
Twitter Timeline Based on
Link List

Submitted to
Dezyne École College
Towards the Partial
Fulfilment of
2nd Semester

By
Pooja Khandelwal



www.dezyneecole.com

Acknowledgement

We take this opportunity to express our sincere gratitude to Dezyne École College for providing us with the knowledge, resources, and guidance essential to the successful completion of this project.

Our deepest thanks to our respected Principal, **Dr. Vinita Mathur**, for her visionary leadership and continuous encouragement. Her commitment to academic excellence and student growth has been a constant source of motivation and pride.

We are equally thankful to **HOD Mr. Tarun Sharma** for his consistent support, expert advice, and valuable insights that shaped our learning and helped us achieve this milestone.

Lastly, we extend heartfelt thanks to everyone who directly or indirectly contributed to the success of this project.

We are proud to be part of Dezyne École College, an institution that truly nurtures professionalism, innovation, and purpose.

With sincere appreciation,

Pooja Khandelwal

Department of BCA

Dezyne École College, Ajmer

Content

Contents

Introduction —What is a Linked List	7
Node Anatomy —	7
Singly vs Doubly	8
Why Not an Array.....	8
Singly Linked List	9
Node Structure & Memory Representation	9
Insert at Head —	10
Insert at Tail —	12
Insert at Position k —	14
Delete Operations — 3 Cases	17
Delete From Head —	17
Delete Tail —	20
Delete Node by Value —	23
Traversal —	26
Search —	28
Count Nodes.....	30
Doubly Linked List	31
Node Structure & Memory Representation	31
Insert at Head —	31
Insert at Tail —	34
Insert at Position —	36
Delete Head —	39
Delete Tail —	42
Delete Node by Value —	45
Forward And Backward Traversal —	48
1. Forward Traversal (Newest → Oldest)	48
2. Backward Traversal (Oldest → Newest)	50
Search —	52
Count Nodes.....	54

Singly LinkList Code –	55
Doubly LinkList Code –	64
Combine LinkList Operations –	76
Full Complexity Table.....	79
When to Use Which.....	80
Summary.....	80

Introduction —What is a Linked List

A Linked List is a linear data structure where elements (called nodes) are NOT stored in contiguous memory locations. Each node holds its data and one or two pointers — memory addresses — that link it to neighbouring nodes. The nodes can live anywhere in RAM; the pointers stitch them into a logical sequence.

Node Anatomy —

Singly Linked List Node —



- Each node: DATA + 1 pointer (next)
- Can ONLY traverse forward (head → tail)
- Tail node: next = NULL

Key difference:

Delete any node

S: $O(n)$ — scan for predecessor
D: $O(1)$ — use node.prev directly

Tail deletion

S: $O(n)$ — scan to 2nd-last
D: $O(1)$ — tail = tail.prev

Doubly Linked List Node —



- Each node: DATA + 2 pointers (prev + next)
- Can traverse BOTH directions
- Head: prev=NULL | Tail: next=NULL

Backward traversal

S: NOT POSSIBLE
D: $O(n)$ via prev pointers

Extra memory

S: Less (1 ptr × n)
D: More (2 ptrs × n)

Real-World Context — Twitter Timeline

Twitter's home timeline shows tweets in reverse-chronological order — newest at the top, oldest at the bottom. Three operations dominate this system:

- **Insert at head:** A new tweet always goes at the very top. This must be instant — $O(1)$.
- **Delete from any position:** Content moderation removes any tweet immediately — $O(1)$.
- **Traverse from head to tail:** Displaying the timeline scrolls from newest to oldest — $O(n)$.

Singly vs Doubly

Property	Singly Linked List	Doubly Linked List
Pointers per node	1 (NEXT only)	2 (PREV + NEXT)
Traversal direction	Forward only (head → tail)	Both directions
Delete tail	$O(n)$ — must scan to 2nd-last	$O(1)$ — use tail.prev
Delete any node (with ptr)	$O(n)$ — scan for predecessor	$O(1)$ — both neighbours known
Extra memory	Less (1 ptr × n)	More (2 ptr × n = +8 bytes/node)
Typical use	Stacks, hash chaining, simple queues	Timelines, browsers, LRU cache, dequeues

Why Not an Array

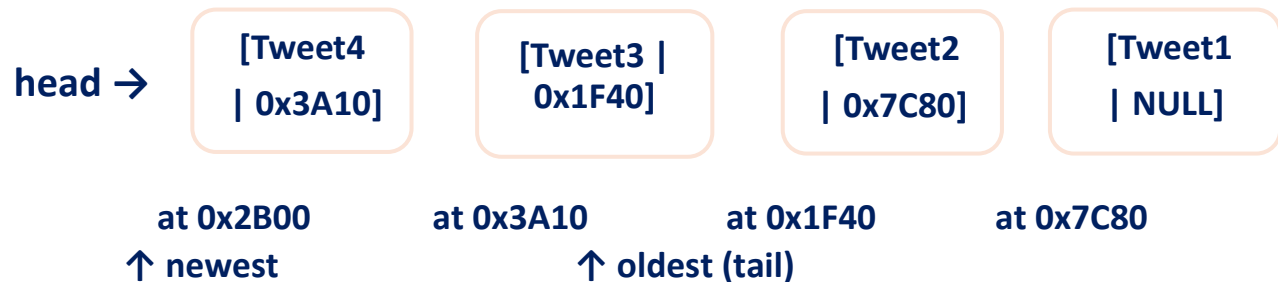
Operation	Array	Linked List	Twitter's Need
Insert at front	$O(n)$ — shift all elements right	$O(1)$ — rewire 2 pointers	New tweet on top — must be instant
Delete any element	$O(n)$ — shift remaining elements	$O(1)$ — rewire neighbours	Remove any tweet instantly
Access by index	$O(1)$ — direct formula	$O(n)$ — traverse from head	Not needed for sequential display
Dynamic size (grow)	Needs reallocation	Trivial — just add a node	Tweet count is unpredictable
Search	$O(n)$ linear, $O(\log n)$ binary	$O(n)$ linear only	Both require traversal for unsorted data

Singly Linked List

Node Structure & Memory Representation

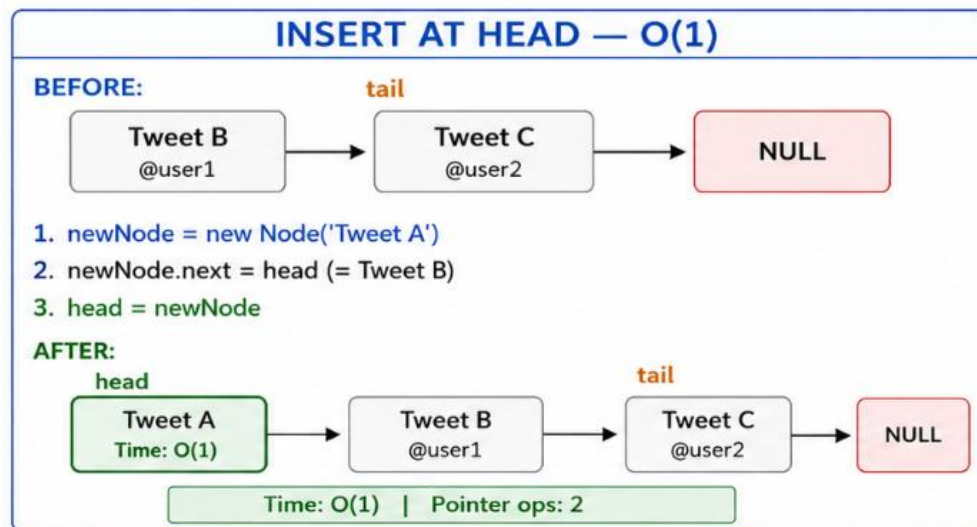
In a singly linked list each node stores data and ONE pointer — next — which holds the memory address of the following node. The last node's next = NULL. We maintain a head pointer to always find the start, and optionally a tail pointer for efficient end-access.

Memory layout example:



Nodes are SCATTERED in RAM — not consecutive like an array.
The only way to reach Tweet2 is: start at head, follow next, follow next, arrive.

Insert at Head —



Concept —

The newest tweet always goes at the top of the timeline. Head insertion achieves this in exactly 2 pointer operations — always $O(1)$ regardless of how many tweets already exist.

Step-by-Step Algorithm

1. **Create new node:** `TweetNode* newNode = new TweetNode(text, user);` — allocates heap memory, sets `next = NULL`.
2. **Connect forward:** `newNode->next = head;` — new node now points to the current head (will become 2nd node).
3. **Move head pointer:** `head = newNode;` — head now refers to the new node.
4. **Handle empty list:** If the list was empty, also set `tail = newNode;`
5. **Increment counter:** `size++;`

Twitter TimeLine Based on Link List

```
cout<<"\n INSERT AT BEGINNING...\n\n";
    userInput();
    NODE *TEMP = new NODE(TID, UName, UTweet);
    if (TEMP == nullptr){
        cout<<"INSUFICIENT MEMORY..TRY AGAIN..\n";
        return 0;
    }
    if (START == nullptr){
        END = START = TEMP;
    }
    else{
        TEMP->NEXT = START;
        START = TEMP;
    }
    countnode++;
    TIDA++;
```

```
// Example trace:
// Before: head → [B] → [C] → NULL
// newNode.next = B (newNode now points to B)
// head = newNode
// After: head → [A] → [B] → [C] → NULL Time: O(1)
```

Why O(1)? No loop, no traversal. The same 2 pointer operations run whether the list has 3 nodes or 3 million nodes.

```
INSERT AT BEGINNING...

Entered TID--TWEET1001

Enter User Name--pooja

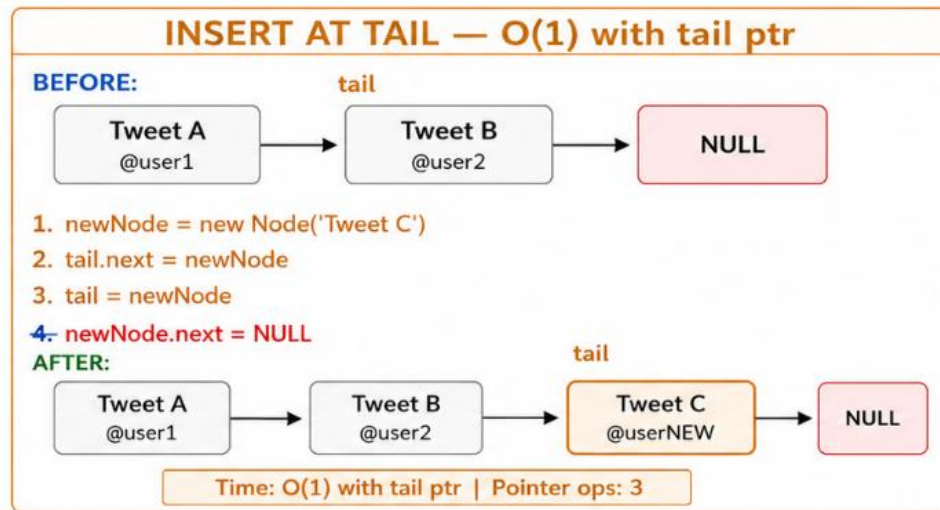
Enter User TWEET--hi pooja
```

```
INSERT AT BEGINNING...

Entered TID--TWEET1001

Enter User Name--pooja
Enter User TWEET--hi pooja
```

Insert at Tail —



Concept —

Without tail pointer: Must traverse all n nodes to reach the last one, then append. Cost = $O(n)$.

With tail pointer : Directly access tail, attach new node, advance tail. Cost = $O(1)$.

Step-by-Step Algorithm (with tail pointer)

1. **Create new node:** `TweetNode* newNode = new TweetNode(text, user);` —
`newNode->next = NULL.`
2. **Handle empty list:** If `head == NULL`: `head = tail = newNode;` return.
3. **Connect:** `tail->next = newNode;` — current tail now points to the new node.
4. **Advance tail:** `tail = newNode;` — tail pointer moves to the new last node.
5. **Increment counter:** `size++;`

Twitter TimeLine Based on Link List

```
cout<<"\n INSERT AT END...\n\n";
    userInput();
    NODE *TEMP = new NODE(TID,UName,UTweet);
    if (TEMP == nullptr){
        cout<<"INSUFICIENT MEMORY..TRY AGAIN..\n";
        return 0;
    }
    if (START == nullptr){
        END = START = TEMP;
    }
    else{
        END->NEXT = TEMP;
        END = TEMP;
    }
    countnode++;
    TIDA++;
```

```
// Example trace:
// Before: head→[A]→[B]→NULL tail=B
// tail.next = newNode (= C)
// tail = C
// After: head→[A]→[B]→[C]→NULL tail=C Time: O(1)
```

```
INSERT AT END...

Entered TID--TWEET1001

Enter User Name--pooja

Enter User TWEET--hi pooja
```

```
INSERT AT END...

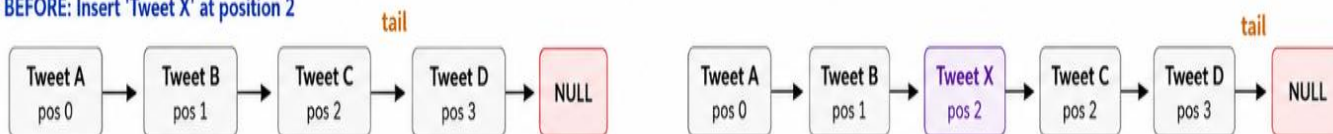
Entered TID--TWEET1001

Enter User Name--pooja
Enter User TWEET--hi pooja|
```

Insert at Position k —

INSERT AT POSITION k — $O(k)$ [traverse k-1 steps, then rewire 2 pointers]

BEFORE: Insert 'Tweet X' at position 2



Steps: 1. curr = head; traverse 1 step to reach pos 1 (Tweet B)
 2. newNode.next = curr.next (= Tweet C at pos 2)
 3. curr.next = newNode

AFTER: [A]→[B]→[X]→[C]→[D]→NULL Time: $O(2)$ traverse + $O(1)$ rewire = $O(k)$

Three sub-cases

- **k = 0:** Same as insertAtHead() — $O(1)$.
- **k = size (last position + 1):** Same as insertAtTail() — $O(1)$.
- **0 < k < size (middle):** Traverse k-1 steps to find the predecessor node, then rewire 2 pointers.

Step-by-Step Algorithm for Middle Insert

Create newNode.

1. Start curr = head; traverse k-1 steps: `for(int i=0; i<k-1; i++) curr = curr->next;`
2. `newNode->next = curr->next;` — new node points to what curr was pointing to.
3. `curr->next = newNode;` — curr now points to new node.
4. **Increment counter:** `size++;`

```
cout<<"\n INSERT AT GIVING POSIOTIN...\n";
    cout<<"Total Nodes -- "<<countnode<<"\n\n";
    userinput();
    int pos;
    cout<<"Enter Position of Node between 0 to "<<countnode<<"---";
    cin>>pos;
    NODE *TEMP = new NODE(TID,UName, UTweet);
    if ( TEMP == nullptr){
        cout<<"Insufficient Memory.. \n";
        return 0;
    }
    int i;
    NODE *TEMP1 = START;
    for ( i = 2 ; i < pos ; i++){
        TEMP1 = TEMP1->NEXT;
    }
    if ( START == nullptr){
        END = START = TEMP;
    }
    else{
        TEMP->NEXT = TEMP1->NEXT;
        TEMP1->NEXT = TEMP;
        if (TEMP1 == END){
            END = TEMP1->NEXT;
        }
    }
    countnode++;
    TIDA++;
```

```
// Example trace:
// Insert 'X' at pos 2 in [A]→[B]→[C]→[D]:
// curr = B (after 1 step, since k-1=1)
// X.next = C ; B.next = X
// Result: [A]→[B]→[X]→[C]→[D] Time: O(2) = O(k)
```

INSERT AT GIVING POSIOTIN...

Total Nodes -- 1

Entered TID--TWEET1002

Enter User Name—pooja

Enter User TWEET--hi pooja

Enter Position of Node between 0 to 1---1

```
INSERT AT GIVING POSIOTIN...
Total Nodes -- 1
Entered TID--TWEET1002
Enter User Name--pooja
Enter User TWEET--hi pooja
Enter Position of Node between 0 to 1---1
```

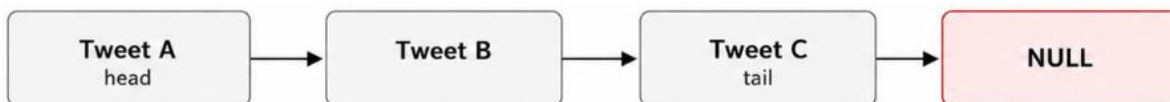
Delete Operations — 3 Cases

Delete From Head —

CASE 1: Delete HEAD — $O(1)$

1. `temp = head`
2. `head = head.next`
3. `delete temp`
4. `if head == NULL: tail = NULL`

BEFORE —



AFTER —



Step-by-Step Algorithm

Moving the head pointer one step forward effectively removes the first node. No traversal needed.

1. **temp = head;** — save the node to free later.
2. **head = head->next;** — head now points to the 2nd node (new head).
3. If head != NULL: it was a single-node list: set **tail = NULL;**
4. **delete temp;** — free heap memory.
5. **size--;**

```
cout<<"\n\n DELETE FROM BEGINNING... \n\n";

    if ( START == nullptr)
    {
        cout<<"\n\nLinkList is Empty.. \n\n";
        return 0;
    }
    else if ( START->NEXT == nullptr){
        TEMP = START;
        END = START = nullptr;
    }
    else{
        TEMP = START;
        START = START->NEXT;
    }
    countnode--;
    delete(TEMP);
    cout<<"\n\n Node Deleted Successfully...\n\n";
```

DELETE OPERATIONS..

1. DELETE FROM BEGINNING
2. DELETE FROM END
3. DELETE FROM GIVEN POSITION
4. Go to Main Menu
5. Exit.

DELETE FROM BEGINNING...

Node Deleted Successfully...

Do You Want..

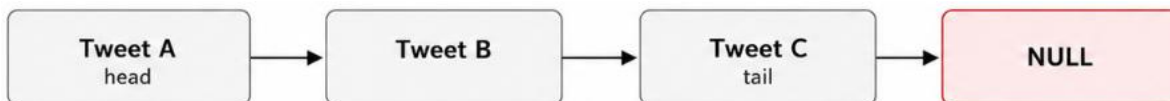
1. Continue With Same
2. Go to Previous Menu
3. Go to Main Menu..
4. Exit

Delete Tail —

CASE 2: Delete TAIL — $O(n)$

1. `curr = head`
2. traverse until `curr.next == tail` → $O(n)$
3. `curr.next = NULL`
4. `tail = curr; delete old_tail`

BEFORE —



AFTER —



Step-by-Step Algorithm

Problem: To delete the tail we must update the second-to-last node's next pointer to NULL. In a singly linked list the only way to find that node is to traverse from head — $O(n)$. This is why doubly linked lists are preferred when tail deletion is frequent.

1. Start **curr = head**; traverse until **curr->next == tail**;
2. **curr->next = NULL**; — second-to-last now terminates the list.
3. **delete tail**; then **tail = curr**;
4. **size--**;

```
cout<<"\n\n DELETE FROM END... \n\n";
    if ( START == nullptr)
    {
        cout<<"\n LinkList is Empty.. \n\n";
        return 0;
    }
    else if ( START->NEXT == nullptr){
        TEMP = START;
        END = START = nullptr;
    }
    else{
        TEMP = START;
        while ( TEMP->NEXT != END)
        {

            TEMP = TEMP->NEXT;
        }
        NODE *T1 = END;
        TEMP->NEXT = nullptr;
        END = TEMP;
        delete(T1);
    }
    countnode--;
    cout<<"\n\n Node Deleted Successfully...\n\n";
```

DELETE OPERATIONS..

1. DELETE FROM BEGINNING
2. DELETE FROM END
3. DELETE FROM GIVEN POSITION
4. Go to Main Menu
5. Exit.

DELETE FROM END...

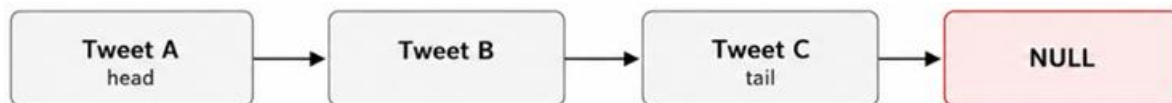
Node Deleted Successfully...

Delete Node by Value —

CASE 3: Delete MIDDLE by value — $O(n)$

1. $prev = NULL, curr = head$
2. while $curr.data \neq target$: $prev = curr, curr = curr.next$
3. $prev.next = curr.next$ (bypass the node)
4. delete $curr$

BEFORE —



AFTER —



Step-by-Step Algorithm

Find the target node and its predecessor simultaneously using two pointers, then bypass the target.

1. **prev = NULL; curr = head;**
2. Traverse: while curr != NULL and curr->data != target: **prev = curr; curr = curr->next;**
3. If curr == NULL: value not found — return.
4. **prev->next = curr->next;** — predecessor skips over the target node.
5. If curr == tail: **tail = prev;** — update tail pointer.
6. **delete curr; size--;**

```
cout<<"\n DELETE FROM GIVEN POSITION... \n\n";
```

```
if ( START == nullptr )
{
    cout<<"\nLinkList is Empty.. \n\n";
    return 0;
}
int pos;
cout<<"Enter Desire Position for Delete 1 - "<<countnode<<"-- ";
cin>>pos;
if ( pos == 1 && START == END )
{
    TEMP = START;
    START = END = nullptr;
}
else{
    TEMP = START;
    for (int i = 2 ; i < pos ; i++)
    {
        TEMP = TEMP->NEXT;
    }
    if (TEMP->NEXT == END){
        NODE *T1 = END;
        TEMP->NEXT = nullptr;
        END = TEMP;
    }
}
```

```
else{
    NODE *T1 = TEMP->NEXT;
    TEMP->NEXT = T1->NEXT;
    delete(T1);
}
}
countnode--;
cout<<"\n Node Deleted Successfully...\n\n";
```

DELETE OPERATIONS..

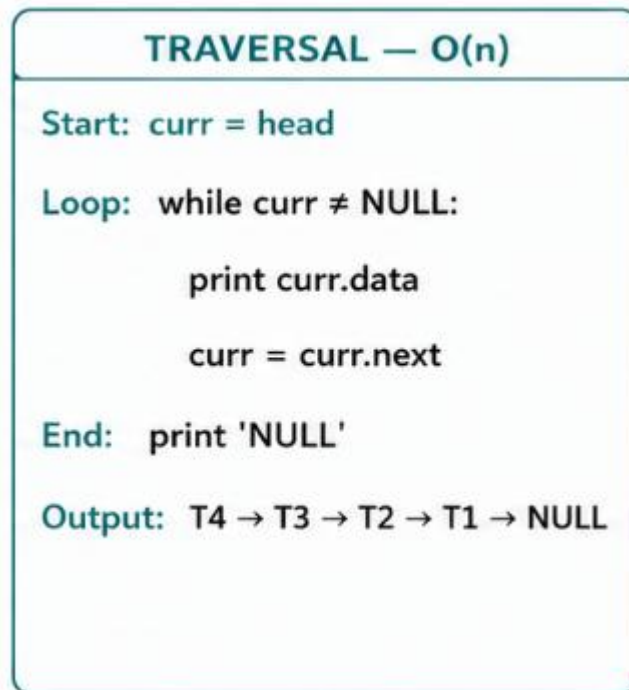
1. DELETE FROM BEGINNING
2. DELETE FROM END
3. DELETE FROM GIVEN POSITION
4. Go to Main Menu
5. Exit.

DELETE FROM GIVEN POSITION...

Enter Desire Position for Delete 1 - 1-- 1

Node Deleted Successfully...

Traversal —



Concept —

Traversal visits every node exactly once, following next pointers from head until NULL. This is how Twitter renders your timeline from top to bottom — each tweet displayed in order.

1. **curr = head;** — start at newest tweet.
2. Loop: **while (curr != nullptr)** — stop when we fall off the end.
3. Inside loop: process curr->data; then **curr = curr->next;**

```
cout<<"\n SINGLY LINKLIST -- \n\n";
cout<<"* Head -> ";
NODE *TEMP = START;
if ( START == nullptr){
    cout<<"NULL \n\nlist is empty\n";
}
else{
    while ( TEMP!= nullptr){
        cout<<"["<<TEMP->ID<<"; ";
        cout<<TEMP->UNAME<<"; ";
        cout<<TEMP->TWEET<<"] -> ";
        TEMP = TEMP->NEXT;
    }
    cout<<"NULL";
}
```

```
##### SINGLY LINKLIST #####
1. Insertion Operation
2. Traversal Operation
3. Delete Operation
4. Search Operation
5. Count Nodes
6. Exit
Go With One Operation --
SINGLY LINKLIST --
* Head -> [TWEET1005, noor, hi ruhi] -> [TWEET1002, krish, hi pooja] -> [TWEET1001, pooja, hi krish] -> NULL
Do you want to Continue .. Press 1 else press any key..]
```

Search —

Concept –

O(n) only: Linked lists have no index formula. To find a node you must start at head and examine each node one by one. Best case $O(1)$ if found at head. Worst case $O(n)$ if at tail or absent.

Binary search is impossible: Binary search requires jumping to the middle element. In a linked list, reaching node i requires following i pointers from head — $O(i)$. So finding the middle costs $O(n/2)$, making binary search $O(n \log n)$ — worse than linear $O(n)$.

```
cout<<"\n\nSearch Tweet in Nodes -- \n\n";
cout<<"* Head -> ";
NODE *TEMP = START;
if ( START == nullptr){
    cout<<"NULL \n\nlist is empty\n";
    return 0;
}
else{
    cout<<"\n\n Enter User Name To Find Tweets -- ";
    string uname;
    int counter = 0;
    getline(cin,uname);
    while ( TEMP!= nullptr){
        if (TEMP->UNAME == uname)
        {
            cout<<"\n["<<TEMP->ID<<" ";
            cout<<TEMP->UNAME<<" ";
            cout<<TEMP->TWEET<<"\n";
            counter++;
        }
    }
}
```

```
##### SINGLY LINKLIST #####

1. Insertion Operation
2. Traversal Operation
3. Delete Operation
4. Search Operation
5. Count Nodes
6. Exit
Go With One Operation --

Search Tweet in Nodes --

* Head ->

Enter User Name To Find Tweets -- pooja
[TWEET1001, pooja, hi krish]
```

Count Nodes

Approach	Algorithm	Time	Recommendation
Traverse and count	curr=head; while curr≠NULL: count++; curr=curr.next	O(n) per call	Avoid for frequent calls
Maintain size counter	size++ on insert, size-- on delete, getCount() returns size	O(1) per call	Always use this approach

```
// Maintain a 'size' member variable in the class
```

```
int getCount() { return size; } // O(1) — just return the variable
```

```
// Increment whenever you insert
```

```
void insertAtHead(string t, string u) { /* ... */; size++; }
```

```
// Decrement whenever you delete
```

```
void deleteHead()
```

```
{ /* ... */; size--; }
```

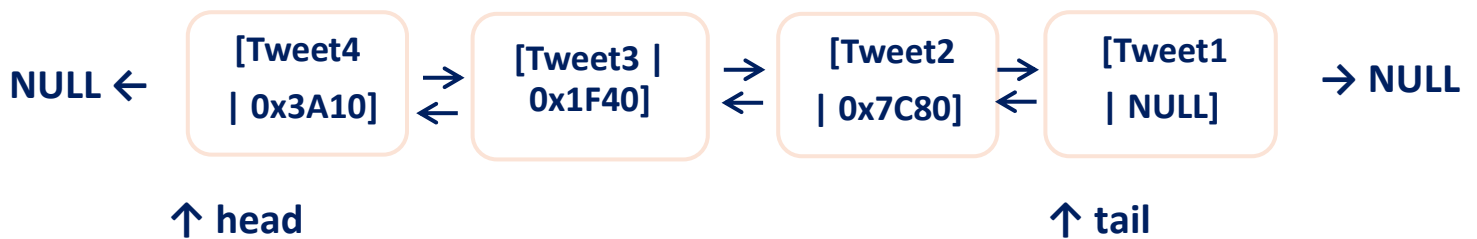
```
void countednode()
{
    system("cls");
    cout<<"Counted Node in LinkList..\n\n";
    cout<<"Total Nodes--"<<countnode<<" Nodes
    Availabed\n";
}
```

```
Counted Node in LinkList..
Total Nodes--3 Nodes Availabed
```

Doubly Linked List

Node Structure & Memory Representation

Memory layout example:



Insert at Head —

BEFORE —



AFTER —



Step-by-Step Algorithm

1. **Create:** `TweetNodeDL* newNode = new TweetNodeDL(text, user);` — prev=NULL, next=NULL.
2. If empty: `head = tail = newNode;` return.
3. **Connect forward:** `newNode->next = head;` — new node points to old head.
4. **Connect backward:** `head->prev = newNode;` — old head points BACK to new node. ← extra step vs singly!
5. **Advance head:** `head = newNode;`
6. `size++;`

```
cout<<"\n INSERT AT BEGINNING...\n\n";
userinput();
NODED *TEMP = new NODED(TID,UName,UTweet);
if (TEMP == nullptr){
    cout<<"INSUFICIENT MEMORY..TRY AGAIN..\n";
    return 0;
}
if (START == nullptr){
    END = START = TEMP;
}
else{
    TEMP->NEXT = START;
    START->PREV = TEMP;
    START = TEMP;
}
countnode++;
TIDA++;
```

```
INSERT AT BEGINNING...  
  
Entered TID--TWEET1001  
  
Enter User Name--krish  
Enter User TWEET--hi ruhi
```

Insert at Tail —

BEFORE —



AFTER —



Step-by-Step Algorithm

1. **Create:** `TweetNodeDL* newNode = new TweetNodeDL(text, user);`
2. If empty: `head = tail = newNode;` return.
3. **Connect backward:** `newNode->prev = tail;` — new node looks BACK at current tail.
4. **Connect forward:** `tail->next = newNode;` — current tail points forward to new node.
5. **Advance tail:** `tail = newNode;`
6. `size++;`

```
cout<<"\n INSERT AT END...\n\n";
    userinput();
    NODED *TEMP = new NODED(TID,UName,UTweet);
    if (TEMP == nullptr){
        cout<<"INSUFICIENT MEMORY..TRY AGAIN..\n";
        return 0;
    }
    if (START == nullptr){
        END = START = TEMP;
    }
    else{
        END->NEXT = TEMP;
        TEMP->PREV = END;
        END = TEMP;
    }
    countnode++;
    TIDA++;
```

```
INSERT AT END...

Entered TID--TWEET1002

Enter User Name--maahi
Enter User TWEET--hi noor
```

Insert at Position —

BEFORE —



AFTER —



Step-by-Step Algorithm

1. **newNode.prev** = A
2. **newNode.next** = B
3. **A.next** = newNode
4. **B.prev** = newNode.
5. **Size++.**

```
cout<<"\n INSERT AT GIVING POSIOTIN...\n";
    cout<<"Total Nodes -- "<<countnode<<"\n\n";
    userinput();
    int pos;
    cout<<"Enter Position of Node between 0 to "<<countnode<<"---";
    cin>>pos;
    NODED *TEMP = new NODED(TID,UName, UTweet);
    if ( TEMP == nullptr){
        cout<<"Insufficient Memory.. \n";
        return 0;
    }
    int i;
    NODED *TEMP1 = START;
    for ( i = 1 ; i < pos ; i++){
        TEMP1 = TEMP1->NEXT;
    }
    if ( START == nullptr){
        END = START = TEMP;
    }
    else{
        TEMP->NEXT = TEMP1->NEXT;
        TEMP1->NEXT = TEMP;
        TEMP->PREV = TEMP1;
        TEMP->NEXT->PREV = TEMP;
        if (TEMP1 == END){
            END = TEMP1->NEXT;
        }
    }
    countnode++;
    TIDA++;
```

```
INSERT AT GIVING POSIOTIN...  
Total Nodes -- 2  
  
Entered TID--TWEET1003  
  
Enter User Name--radhe  
Enter User TWEET--hi krish  
Enter Position of Node between 0 to 2---1
```

Delete Head —

CASE 1: Delete HEAD — $O(1)$

1. `head = head.next`
2. `head.prev = NULL` ← new head has no pred
3. `delete old_head`
4. `if head==NULL: tail=NULL`

BEFORE —



AFTER —



Step-by-Step Algorithm

1. **Check:** If head == NULL: empty list, return.
2. **Save:** TweetNodeDL* temp = head;
3. **Advance head:** head = head->next;
4. **If head != NULL:** head->prev = nullptr; — new head has no predecessor.
5. **Else (list now empty):** tail = nullptr;
6. **Free:** delete temp;
7. **Size--;**

```
cout<<"\n\n DELETE FROM BEGINNING... \n\n";
    if ( START == nullptr)
    {
        cout<<"\n\nLinkList is Empty.. \n\n";
        return 0;
    }
    else if ( START->NEXT == nullptr){
        TEMP = START;
        END = START = nullptr;
    }
    else{
        TEMP = START;
        START = START->NEXT;
        START->PREV = nullptr;
    }
    countnode--;
    delete(TEMP);
    cout<<"\n\n Node Deleted Successfully...\n\n";
```

DELETE OPERATIONS..

1. DELETE FROM BEGINNING
2. DELETE FROM END
3. DELETE FROM GIVEN POSITION
4. Go to Main Menu
5. Exit.

DELETE FROM BEGINNING...

Node Deleted Successfully...

Do You Want..

1. Continue With Same
2. Go to Previous Menu
3. Go to Main Menu..
4. Exit

Delete Tail —

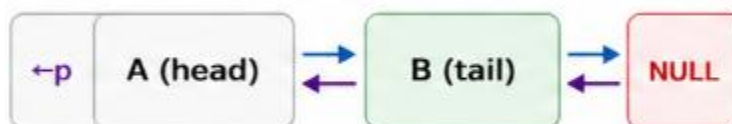
CASE 2: Delete TAIL — $O(1)$ ← KEY WIN!

1. $\text{tail} = \text{tail.prev} \leftarrow O(1)$ via prev pointer
2. $\text{tail.next} = \text{NULL}$
3. delete old_tail
4. if $\text{tail} == \text{NULL}$: $\text{head} = \text{NULL}$

BEFORE —



AFTER —



Step-by-Step Algorithm

8. **Check:** If tail == NULL: empty list, return.
9. **Save:** TweetNodeDL* temp = tail;
10. **Advance tail BACKWARD:** tail = tail->prev; ONLY possible because of the prev pointer.
11. If tail != NULL: tail->next = nullptr; — new tail has no successor.
12. Else (list now empty): head = nullptr;
13. delete temp;
14. Size--;

```
cout<<"\n\n DELETE FROM END... \n\n";
if ( START == nullptr)
{
    cout<<"\n LinkList is Empty.. \n\n";
    return 0;
}
else if ( START->NEXT == nullptr){
    TEMP = START;
    END = START = nullptr;
}
else{
    TEMP = START;
    while ( TEMP->NEXT != END)
    {

        TEMP = TEMP->NEXT;
    }
    NODED *T1 = END;
    TEMP = TEMP->PREV;
    TEMP->NEXT = nullptr;
    END = TEMP;
    delete(T1);
}
countnode--;
cout<<"\n\n Node Deleted Successfully...\n\n";
```

DELETE OPERATIONS..

1. DELETE FROM BEGINNING
2. DELETE FROM END
3. DELETE FROM GIVEN POSITION
4. Go to Main Menu
5. Exit.

DELETE FROM END...

Node Deleted Successfully...

Delete Node by Value –

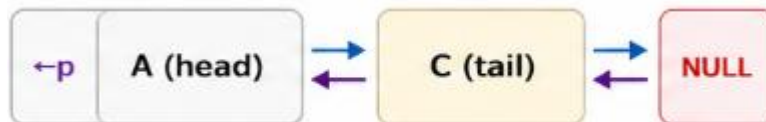
CASE 3: Delete MIDDLE — O(1) with ptr

1. $N.\text{prev}.\text{next} = N.\text{next}$ (A skips over N)
 2. $N.\text{next}.\text{prev} = N.\text{prev}$ (B points back to A)
 3. delete N
- (Only 2 pointer rewires!)

BEFORE –



AFTER –



Step-by-Step Algorithm

1. **Check edge cases:** if $N == \text{head}$ → call `deleteHead()`; if $N == \text{tail}$ → call `deleteTail()`.
2. **A.next = N.next:** $N \rightarrow \text{prev} \rightarrow \text{next} = N \rightarrow \text{next}$; — predecessor skips over N.
3. **B.prev = N.prev:** $N \rightarrow \text{next} \rightarrow \text{prev} = N \rightarrow \text{prev}$; — successor points back past N.
4. delete N;
5. Size--;

```
cout<<"\n DELETE FROM GIVEN POSITION... \n\n";

if ( START == nullptr )
{
    cout<<"\nLinkList is Empty.. \n\n";
    return 0;
}
int pos;
cout<<"Enter Desire Position for Delete 1 - "<<countnode<<"-- ";
cin>>pos;
if ( pos == 1 && START == END )
{
    TEMP = START;
    START = nullptr;
    END = nullptr;
}
else{
    TEMP = START;
    for (int i = 1 ; i < pos ; i++)
    {
        TEMP = TEMP->NEXT;
    }
    if (TEMP == END){
        TEMP->PREV->NEXT = nullptr;
        END = TEMP->PREV;
    }
    else{
        TEMP->PREV->NEXT = TEMP->NEXT;
        TEMP->NEXT->PREV = TEMP->PREV;
        delete(TEMP);
    }
}
countnode--;
cout<<"\n Node Deleted Successfully...\n\n";
```

DELETE OPERATIONS..

1. DELETE FROM BEGINNING
2. DELETE FROM END
3. DELETE FROM GIVEN POSITION
4. Go to Main Menu
5. Exit.

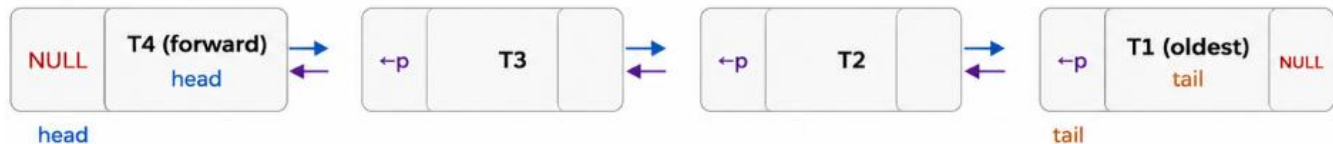
DELETE FROM GIVEN POSITION...

Enter Desire Position for Delete 1 - 2-- 2

Node Deleted Successfully...

Forward And Backward Traversal —

The Doubly Link List —



1. Forward Traversal (Newest → Oldest)

FORWARD TRAVERSAL → $O(n)$

```
Start: curr = head
Loop: while curr ≠ NULL:
    print curr.data
    curr = curr.next ← follow next
End: reach NULL (stop)
```

Output (newest → oldest):
T4 → T3 → T2 → T1 → NULL

Time: $O(n)$ | Space: $O(1)$

```
cout<<"\n FORWARD TRAVERSAL--\n\n";
cout<<"NULL<-";
NODED *TEMP = START;
while ( TEMP!= nullptr){
    cout<<"["<<TEMP->ID<<"; ";
    cout<<TEMP->UNAME<<"; ";
    cout<<TEMP->TWEET<<"] -> ";
    TEMP = TEMP->NEXT;
}
cout<<"NULL\n\n";
```

```
DOUBLY LINKLIST --
-
1. FORWARD TRAVERSAL--
2. BACKWARD TRAVERSAL--
3. Go To Main Menu--
4. Exit--

FORWARD TRAVERSAL--
NULL<-[TWEET1003, radhe, hi krish] -> NULL
```

2. Backward Traversal (Oldest → Newest)

BACKWARD TRAVERSAL ← $O(n)$

Start: $\text{curr} = \text{tail}$

Loop: while $\text{curr} \neq \text{NULL}$:

 print curr.data

$\text{curr} = \text{curr.prev} \leftarrow \text{follow prev}$

End: reach NULL (stop)

Output (oldest → newest):

$T1 \rightarrow T2 \rightarrow T3 \rightarrow T4 \rightarrow \text{NULL}$

Time: $O(n)$ | Space: $O(1)$

```
cout<<"\n BACKWARD TRAVERSAL--\n\n";
    cout<<"NULL<-";
    NODED *TEMP = END;
    while ( TEMP!= nullptr){
        cout<<"["<<TEMP->ID<<" , ";
        cout<<TEMP->UNAME<<" , ";
        cout<<TEMP->TWEET<<" ] -> ";
        TEMP = TEMP->PREV;
    }
    cout<<"NULL\n\n";
```

```
so with one operation
DOUBLY LINKLIST --

1. FORWARD TRAVERSAL--
2. BACKWARD TRAVERSAL--
3. Go To Main Menu--
4. Exit--

BACKWARD TRAVERSAL--

NULL<-[TWEET1003, radhe, hi krish] -> NULL
```

Real-World Applications of Backward Traversal

Application	How Backward Traversal is Used
Browser back button	Navigate history in reverse (most recent → oldest visited)
Text editor undo/redo	Move backward through change history to undo actions
Music player Previous Track	Go backward through the playlist
LRU Cache eviction	Tail is LRU item — accessed via backward from end, evicted O(1)
Twitter / Social Media	Show thread in chronological order (oldest reply first)

Search —

Concept –

O(n) only: Linked lists have no index formula. To find a node you must start at head and examine each node one by one. Best case O(1) if found at head. Worst case O(n) if at tail or absent.

Binary search is impossible: Binary search requires jumping to the middle element. In a linked list, reaching node i requires following i pointers from head — O(i). So finding the middle costs O(n/2), making binary search O(n log n) — worse than linear O(n).

```
cout<<"\n\nSearch Tweet in Nodes -- \n\n";
cout<<"* Head -> ";
NODE *TEMP = START;
if ( START == nullptr){
    cout<<"NULL \n\nlist is empty\n";
    return 0;
}
else{
    cout<<"\n\n Enter User Name To Find Tweets -- ";
    string uname;
    int counter = 0;
    getline(cin,uname);
    while ( TEMP!= nullptr){
        if (TEMP->UNAME == uname)
        {
            cout<<"\n["<<TEMP->ID<<" ";
            cout<<TEMP->UNAME<<" ";
            cout<<TEMP->TWEET<<"\n";
            counter++;
        }
        TEMP = TEMP->NEXT;
    }
    if ( counter == 0){
        cout<<"\n No Tweet Found..\n\n";
    }
}
```

```
##### SINGLY LINKLIST #####

1. Insertion Operation
2. Traversal Operation
3. Delete Operation
4. Search Operation
5. Count Nodes
6. Exit
Go With One Operation --

Search Tweet in Nodes --

* Head ->

Enter User Name To Find Tweets -- pooja
[TWEET1001, pooja, hi krish]
```

Count Nodes

Approach	Algorithm	Time	Recommendation
Traverse and count	curr=head; while curr≠NULL: count++; curr=curr.next	O(n) per call	Avoid for frequent calls
Maintain size counter	size++ on insert, size-- on delete, getCount() returns size	O(1) per call	Always use this approach

```
// Maintain a 'size' member variable in the class
```

```
int getCount() { return size; } // O(1) — just return the variable
```

```
// Increment whenever you insert
```

```
void insertAtHead(string t, string u) { /* ... */; size++; }
```

```
// Decrement whenever you delete
```

```
void deleteHead()
```

```
{ /* ... */; size--; }
```

```
void countednode()
{
    system("cls");
    cout<<"Counted Node in LinkList..\n\n";
    cout<<"Total Nodes--"<<countnode<<" Nodes
    Availabed\n";
}
```

```
Counted Node in LinkList..
Total Nodes--3 Nodes Availabed
```

Singly LinkList Code –

```
#include<iostream>
#include<string>
#include<conio.h>
using namespace std;
class NODE
{
public:
    string ID, UNAME, TWEET;
    NODE *NEXT;
    NODE( string x, string y, string z)
    {
        ID = x;
        UNAME = y;
        TWEET = z;
        NEXT = nullptr;
    }
};
class SLinkList
{
public:
    string TID, UName, UTweet;
    int TIDA, countnode;
    NODE *START, *END;
    SLinkList()
    {
        TIDA = 1001;
        countnode = 0;
        START = nullptr;
        END = nullptr;
    }
    void userinput()
    {
        TID = "TWEET" + to_string(TIDA);
        cout<<"Entered TID--"<<TID<<endl;
        cin.ignore();
        cout<<"Enter User Name--";
        getline(cin,UName);
        cout<<"Enter User TWEET--";
        getline(cin,UTweet);

    }
    void countednode()
```

Twitter TimeLine Based on Link List

```
{
    system("cls");
    cout<<"Counted Node in LinkList..\n\n";
    cout<<"Total Nodes--"<<countnode<<" Nodes Availabed\n";
}
char continuecheck()
{
    char c;
    cout<<"\n\nDo You Want.. \n";
    cout<<"1. Continue With Same\n";
    cout<<"2. Go to Previous Menu\n";
    cout<<"3. Go to Main Menu..\n";
    cout<<"4. Exit\n";
    c = getch();
    if ( c == '4'){
        thanksLine();
        exit(0);
    }
    else{
        return (c);
    }
}
```

```
int insertnode(int opr)
{
    system("cls");
    if( opr == 1)
    {
        cout<<"\n INSERT AT BEGINNING...\n\n";
        userinput();
        NODE *TEMP = new NODE(TID,UName,UTweet);
        if (TEMP == nullptr){
            cout<<"INSUFICIENT MEMORY..TRY AGAIN..\n";
            return 0;
        }
        if (START == nullptr){
            END = START = TEMP;
        }
        else{
            TEMP->NEXT = START;
            START = TEMP;
        }
        countnode++;
        TIDA++;
    }
}
```

```
}
else if( opr == 2){
    cout<<"\n INSERT AT END...\n\n";
    userinput();
    NODE *TEMP = new NODE(TID,UName,UTweet);
    if (TEMP == nullptr){
        cout<<"INSUFICIENT MEMORY..TRY AGAIN..\n";
        return 0;
    }
    if (START == nullptr){
        END = START = TEMP;
    }
    else{
        END->NEXT = TEMP;
        END = TEMP;
    }
    countnode++;
    TIDA++;
}
else if (opr = 3){
    cout<<"\n INSERT AT GIVING POSIOTIN...\n";
    cout<<"Total Nodes -- "<<countnode<<"\n\n";
    userinput();
    int pos;
    cout<<"Enter Position of Node between 0 to "<<countnode<<"---";
    cin>>pos;
    NODE *TEMP = new NODE(TID,UName, UTweet);
    if ( TEMP == nullptr){
        cout<<"Insufficient Memory.. \n";
        return 0;
    }
    int i;
    NODE *TEMP1 = START;
    for ( i = 2 ; i < pos ; i++){
        TEMP1 = TEMP1->NEXT;
    }
    if ( START == nullptr){
        END = START = TEMP;
    }
    else{
        TEMP->NEXT = TEMP1->NEXT;
        TEMP1->NEXT = TEMP;
        if (TEMP1 == END){
            END = TEMP1->NEXT;
        }
    }
}
```

```
        countnode++;
        TIDA++;
    }
    char c = continuecheck();
    if (c == '1')
        insertnode(opr);
    else if ( c == '2')
        insertionoperation();
    else if ( c == '3')
        mainmenu();
}

int insertionoperation()
{
    system("cls");
    cout<<"INSERTION OPERATION\n\n";
    cout<<"1. Insert At Beginning\n";
    cout<<"2. Insert At End\n";
    cout<<"3. Insert At Giving Position\n";
    char c = getch();
    if (c == '1'){
        insertnode(1);
    }
    else if (c == '2'){
        insertnode(2);

    }
    else if (c == '3'){
        insertnode(3);
    }
    else if (c == '4'){
        mainmenu();
    }
    else if (c == '5'){
        thanksLine();
        exit(0);
    }
}

void traverselist()
{
    cout<<"\n SINGLY LINKLIST -- \n\n";
    cout<<"* Head -> ";
    NODE *TEMP = START;
    if ( START == nullptr){
        cout<<"NULL \n\nlist is empty\n";
    }
    else{
```

Twitter TimeLine Based on Link List

```
while ( TEMP!= nullptr){
    cout<<"["<<TEMP->ID<<", ";
    cout<<TEMP->UNAME<<", ";
    cout<<TEMP->TWEET<<"] -> ";
    TEMP = TEMP->NEXT;
}
cout<<"NULL";
}
}

int searchtweetinnode()
{
cout<<"\n\nSearch Tweet in Nodes -- \n\n";
cout<<"* Head -> ";
NODE *TEMP = START;
if ( START == nullptr){
    cout<<"NULL \n\nlist is empty\n";
    return 0;
}
else{
    cout<<"\n\n Enter User Name To Find Tweets -- ";
    string uname;
    int counter = 0;
    getline(cin,uname);
    while ( TEMP!= nullptr){
        if (TEMP->UNAME == uname)
        {
            cout<<"\n["<<TEMP->ID<<", ";
            cout<<TEMP->UNAME<<", ";
            cout<<TEMP->TWEET<<"]\n";
            counter++;
        }
        TEMP = TEMP->NEXT;
    }
    if ( counter == 0){
        cout<<"\n No Tweet Found..\n\n";
    }
}
char c;
cout<<"\n\n Do You Want To .. \n";
cout<<"1. Continue With Same\n";
cout<<"2. Go To Main Menu..\n";
cout<<"3. Exit\n";
c = getch();
if (c == '1')
    searchtweetinnode();
else if ( c == '2')
```

Twitter TimeLine Based on Link List

```
    mainmenu();
else if ( c == '3'){
    thanksLine();
    exit(0);
}
}
int deletenode(int opr)
{
    NODE *TEMP;
    if ( opr == 1){
        cout<<"\n\n DELETE FROM BEGINNING... \n\n";

        if ( START == nullptr)
        {
            cout<<"\n\nLinkList is Empty.. \n\n";
            return 0;
        }
        else if ( START->NEXT == nullptr){
            TEMP = START;
            END = START = nullptr;
        }
        else{
            TEMP = START;
            START = START->NEXT;
        }
        countnode--;
        delete(TEMP);
        cout<<"\n\n Node Deleted Successfully...\n\n";
    }
    else if ( opr == 2){
        cout<<"\n\n DELETE FROM END... \n\n";
        if ( START == nullptr)
        {
            cout<<"\n LinkList is Empty.. \n\n";
            return 0;
        }
        else if ( START->NEXT == nullptr){
            TEMP = START;
            END = START = nullptr;
        }
        else{
            TEMP = START;
            while ( TEMP->NEXT != END)
            {

                TEMP = TEMP->NEXT;
```

Twitter TimeLine Based on Link List

```
    }
    NODE *T1 = END;
    TEMP->NEXT = nullptr;
    END = TEMP;
    delete(T1);
}
countnode--;
cout<<"\n\n Node Deleted Successfully...\n\n";
}
else if ( opr == 3){
    cout<<"\n DELETE FROM GIVEN POSITION... \n\n";

    if ( START == nullptr )
    {
        cout<<"\nLinkList is Empty.. \n\n";
        return 0;
    }
    int pos;
    cout<<"Enter Desire Position for Delete 1 - "<<countnode<<"-- ";
    cin>>pos;
    if ( pos == 1 && START == END )
    {
        TEMP = START;
        START = END = nullptr;
    }
    else{
        TEMP = START;
        for (int i = 2 ; i < pos ; i++)
        {
            TEMP = TEMP->NEXT;
        }
        if (TEMP->NEXT == END){
            NODE *T1 = END;
            TEMP->NEXT = nullptr;
            END = TEMP;
        }
        else{
            NODE *T1 = TEMP->NEXT;
            TEMP->NEXT = T1->NEXT;
            delete(T1);
        }
    }
    countnode--;
    cout<<"\n Node Deleted Successfully...\n\n";
}
char c = continuecheck();
```

Twitter TimeLine Based on Link List

```
    if (c == '1')
        deletenode(opr);
    else if ( c == '2')
        deleteoperation();
    else if ( c == '3')
        mainmenu();
}

int deleteoperation()
{
    system("cls");
    cout<<"\n DELETE OPERATIONS..\n\n";
    cout<<"1. DELETE FROM BEGINNING\n";
    cout<<"2. DELETE FROM END\n";
    cout<<"3. DELETE FROM GIVEN POSITION\n";
    cout<<"4. Go to Main Menu\n";
    cout<<"5. Exit.\n";
    char c = getch();
    if ( c == '1'){
        deletenode(1);
    }
    else if ( c == '2'){
        deletenode(2);
    }
    else if ( c == '3'){
        deletenode(3);
    }
    else if ( c == '4'){
        mainmenu();
    }
    else if ( c == '5'){
        thanksLine();
        exit(0);
    }
}

void mainmenu()
{
    system("cls");
    cout<<"##### SINGLY LINKLIST #####\n\n";
    cout<<"1. Insertion Operation\n";
    cout<<"2. Traversal Operation\n";
    cout<<"3. Delete Operation\n";
    cout<<"4. Search Operation\n";
    cout<<"5. Count Nodes\n";
}
```

Twitter TimeLine Based on Link List

```
    cout<<"6. Exit\n";
    cout<<"Go With One Operation -- ";
    char c = getch();
    if ( c == '1'){
        insertionoperation();
    }
    else if ( c == '2'){
        traverselist();
    }
    else if ( c == '3'){
        deleteoperation();

    }
    else if ( c == '4'){
        searchtweetinnode();
    }
    else if ( c == '5'){
        countednode();

    }
    else if ( c == '6'){
        thanksLine();
        exit(0);
    }
    cout<<"\n\n Do you want to Continue .. Press 1 else press any key..";
    c = getch();
    if ( c == '1'){
        mainmenu();
    }
}
void thanksLine()
{
    cout<<"\n\n Thank You for Using My App...\n\n";
}

};
/*int main()
{
    SLinkedList s;
    s.mainmenu();
    cout<<"\n\n Thank You For Using My App...\n\n";
}*/
```

Doubly LinkList Code –

```
#include<iostream>
#include<string>
#include<conio.h>
using namespace std;
class NODED
{
public:
    string ID, UNAME, TWEET;
    NODED *PREV ,*NEXT;
    NODED( string x, string y, string z)
    {
        PREV = nullptr;
        ID = x;
        UNAME = y;
        TWEET = z;
        NEXT = nullptr;
    }
};
class DLinkedList
{
public:
    string TID, UName, UTweet;
    int TIDA, countnode;
    NODED *START, *END;
    DLinkedList()
    {
        TIDA = 1001;
        countnode = 0;
        START = END = nullptr;
    }
    void userinput()
    {
        TID = "TWEET" + to_string(TIDA);
        cout<<"Entered TID--"<<TID<<endl;
        cin.ignore();
        cout<<"Enter User Name--";
```

Twitter TimeLine Based on Link List

```
    getline(cin,UName);
    cout<<"Enter User TWEET--";
    getline(cin,UTweet);

}

void countednode()
{
    system("cls");
    cout<<"Counted Node in LinkList..\n\n";
    cout<<"Total Nodes--"<<countnode<<" Nodes Availabed\n";
}

char continuecheck()
{
    char c;
    cout<<"\n\nDo You Want.. \n";
    cout<<"1. Continue With Same\n";
    cout<<"2. Go to Previous Menu\n";
    cout<<"3. Go to Main Menu..\n";
    cout<<"4. Exit\n";
    c = getch();
    if ( c == '4'){
        thanksLine();
        exit(0);
    }
    else{
        return (c);
    }
}

int insertnode(int opr)
{
    system("cls");
    if( opr == 1)
    {
        cout<<"\n INSERT AT BEGINNING...\n\n";
        userinput();
        NODED *TEMP = new NODED(TID,UName,UTweet);
```

Twitter TimeLine Based on Link List

```
    if (TEMP == nullptr){
        cout<<"INSUFICIENT MEMORY..TRY AGAIN..\n";
        return 0;
    }
    if (START == nullptr){
        END = START = TEMP;
    }
    else{
        TEMP->NEXT = START;
        START->PREV = TEMP;
        START = TEMP;
    }
    countnode++;
    TIDA++;
}
else if( opr == 2){
    cout<<"\n INSERT AT END...\n\n";
    userInput();
    NODED *TEMP = new NODED(TID,UName,UTweet);
    if (TEMP == nullptr){
        cout<<"INSUFICIENT MEMORY..TRY AGAIN..\n";
        return 0;
    }
    if (START == nullptr){
        END = START = TEMP;
    }
    else{
        END->NEXT = TEMP;
        TEMP->PREV = END;
        END = TEMP;
    }
    countnode++;
    TIDA++;
}
else if (opr = 3){
    cout<<"\n INSERT AT GIVING POSIOTIN...\n";
    cout<<"Total Nodes -- "<<countnode<<"\n\n";
    userInput();
    int pos;
```

Twitter TimeLine Based on Link List

```
    cout<<"Enter Position of Node between 0 to "<<countnode<<"---";
    cin>>pos;
    NODED *TEMP = new NODED(TID,UName, UTweet);
    if ( TEMP == nullptr){
        cout<<"Insufficient Memory.. \n";
        return 0;
    }
    int i;
    NODED *TEMP1 = START;
    for ( i = 1 ; i < pos ; i++){
        TEMP1 = TEMP1->NEXT;
    }
    if ( START == nullptr){
        END = START = TEMP;
    }
    else{
        TEMP->NEXT = TEMP1->NEXT;
        TEMP1->NEXT = TEMP;
        TEMP->PREV = TEMP1;
        TEMP->NEXT->PREV = TEMP;
        if (TEMP1 == END){
            END = TEMP1->NEXT;
        }

    }
    countnode++;
    TIDA++;
}
char c = continuecheck();
if (c == '1')
    insertnode(opr);
else if ( c == '2')
    insertionoperation();
else if ( c == '3'){
    mainmenu();
}
return 0;
}

void insertionoperation()
```

Twitter TimeLine Based on Link List

```
{
    system("cls");
    cout<<"INSERTION OPERATION\n\n";
    cout<<"1. Insert At Beginning\n";
    cout<<"2. Insert At End\n";
    cout<<"3. Insert At Giving Position\n";
    char c = getch();
    if (c == '1'){
        insertnode(1);
    }
    else if (c == '2'){
        insertnode(2);
    }
    else if (c == '3'){
        insertnode(3);
    }
    else if (c == '4'){
        mainmenu();
    }
    else if (c == '5'){
        thanksLine();
        exit(0);
    }
}

int traverselist()
{
    cout<<"\n DOUBLY LINKLIST -- \n\n";
    if ( START == nullptr){
        cout<<"NULL \n\nlist is empty\n";
        return 0;
    }
    cout<<"\n 1. FORWARD TRAVERSAL--\n";
    cout<<"\n 2. BACKWARD TRAVERSAL--\n";
    cout<<"\n 3. Go To Main Menu--\n";
    cout<<"\n 4. Exit--\n";
    char c = getch();
    if( c == '1'){
        cout<<"\n FORWARD TRAVERSAL--\n\n";
```

Twitter TimeLine Based on Link List

```
cout<<"NULL<-";
NODED *TEMP = START;
while ( TEMP!= nullptr){
    cout<<"["<<TEMP->ID<<"," ";
    cout<<TEMP->UNAME<<"," ";
    cout<<TEMP->TWEET<<"] -> ";
    TEMP = TEMP->NEXT;
}
cout<<"NULL\n\n";
}
else if ( c == '2' ){
    cout<<"\n BACKWARD TRAVERSAL--\n\n";
    cout<<"NULL<-";
    NODED *TEMP = END;
    while ( TEMP!= nullptr){
        cout<<"["<<TEMP->ID<<"," ";
        cout<<TEMP->UNAME<<"," ";
        cout<<TEMP->TWEET<<"] -> ";
        TEMP = TEMP->PREV;
    }
    cout<<"NULL\n\n";
}
else if (c == '3'){
    mainmenu();
}
else if (c == '4'){
    thanksLine();
    exit(0);
}
return 0;
}

int searchtweetinnode()
{
    cout<<"\n\nSearch Tweet in Nodes -- \n\n";
    cout<<"* Head -> ";
    NODED *TEMP = START;
    if ( START == nullptr){
        cout<<"NULL \n\nlist is empty\n";
        return 0;
    }
}
```

Twitter TimeLine Based on Link List

```
}
else{
    cout<<"\n\n Enter User Name To Find Tweets -- ";
    string uname;
    int counter = 0;
    getline(cin,uname);
    while ( TEMP!= nullptr){
        if (TEMP->UNAME == uname)
        {
            cout<<"\n["<<TEMP->ID<<" , ";
            cout<<TEMP->UNAME<<" , ";
            cout<<TEMP->TWEET<<"]\n";
            counter++;
        }
        TEMP = TEMP->NEXT;
    }
    if ( counter == 0){
        cout<<"\n No Tweet Found..\n\n";
    }
}
char c;
cout<<"\n\n Do You Want To .. \n";
cout<<"1. Continue With Same\n";
cout<<"2. Go To Main Menu..\n";
cout<<"3. Exit\n";
c = getch();
if (c == '1')
    searchtweetinnode();
else if ( c == '2')
    mainmenu();
else if ( c == '3'){
    thanksLine();
    exit(0);
}
return 0;
}
int deletenode(int opr)
{
    NODED *TEMP;
```

```
if ( opr == 1){
    cout<<"\n\n DELETE FROM BEGINNING... \n\n";
    if ( START == nullptr)
    {
        cout<<"\n\nLinkList is Empty.. \n\n";
        return 0;
    }
    else if ( START->NEXT == nullptr){
        TEMP = START;
        END = START = nullptr;
    }
    else{
        TEMP = START;
        START = START->NEXT;
        START->PREV = nullptr;
    }
    countnode--;
    delete(TEMP);
    cout<<"\n\n Node Deleted Successfully...\n\n";
}
else if ( opr == 2){
    cout<<"\n\n DELETE FROM END... \n\n";
    if ( START == nullptr)
    {
        cout<<"\n LinkList is Empty.. \n\n";
        return 0;
    }
    else if ( START->NEXT == nullptr){
        TEMP = START;
        END = START = nullptr;
    }
    else{
        TEMP = START;
        while ( TEMP->NEXT != END)
        {

            TEMP = TEMP->NEXT;
        }
        NODED *T1 = END;
```

Twitter TimeLine Based on Link List

```
    TEMP = TEMP->PREV;
    TEMP->NEXT = nullptr;
    END = TEMP;
    delete(T1);
}
countnode--;
cout<<"\n\n Node Deleted Successfully...\n\n";
}
else if ( opr == 3){
    cout<<"\n DELETE FROM GIVEN POSITION... \n\n";

    if ( START == nullptr )
    {
        cout<<"\nLinkList is Empty.. \n\n";
        return 0;
    }
    int pos;
    cout<<"Enter Desire Position for Delete 1 - "<<countnode<<"-- ";
    cin>>pos;
    if ( pos == 1 && START == END )
    {
        TEMP = START;
        START = nullptr;
        END = nullptr;
    }
    else{
        TEMP = START;
        for (int i = 1 ; i < pos ; i++)
        {
            TEMP = TEMP->NEXT;
        }
        if (TEMP == END){
            TEMP->PREV->NEXT = nullptr;
            END = TEMP->PREV;
        }
        else{
            TEMP->PREV->NEXT = TEMP->NEXT;
            TEMP->NEXT->PREV = TEMP->PREV;
            delete(TEMP);
        }
    }
}
```

```
    }
}
countnode--;
cout<<"\n Node Deleted Successfully...\n\n";
}
char c = continuecheck();
if (c == '1')
    deletenode(opr);
else if ( c == '2')
    deleteoperation();
else if ( c == '3')
    mainmenu();
return 0;
}

void deleteoperation()
{
    system("cls");
    cout<<"\n DELETE OPERATIONS..\n\n";
    cout<<"1. DELETE FROM BEGINNING\n";
    cout<<"2. DELETE FROM END\n";
    cout<<"3. DELETE FROM GIVEN POSITION\n";
    cout<<"4. Go to Main Menu\n";
    cout<<"5. Exit.\n";
    char c = getch();
    if ( c == '1'){
        deletenode(1);
    }
    else if ( c == '2'){
        deletenode(2);
    }
    else if ( c == '3'){
        deletenode(3);
    }
    else if ( c == '4'){
        mainmenu();
    }
    else if ( c == '5'){
        thanksLine();
    }
}
```

```
        exit(0);
    }
}

void mainmenu()
{
    system("cls");
    cout<<"##### DOUBLY LINKLIST #####\n\n";
    cout<<"1. Insertion Operation\n";
    cout<<"2. Traversal Operation\n";
    cout<<"3. Delete Operation\n";
    cout<<"4. Search Operation\n";
    cout<<"5. Count Nodes\n";
    cout<<"6. Exit\n";
    cout<<"Go With One Operation -- ";
    char c = getch();
    if ( c == '1'){
        insertionoperation();
    }
    else if ( c == '2'){
        traverselist();
    }
    else if ( c == '3'){
        deleteoperation();

    }
    else if ( c == '4'){
        searchtweetinnode();
    }
    else if ( c == '5'){
        countednode();

    }
    else if ( c == '6'){
        thanksLine();
        exit(0);
    }
    cout<<"\n\n Do you want to Continue .. Press 1 else press any key..";
}
```

Twitter TimeLine Based on Link List

```
    c = getch();
    if ( c == '1'){
        mainmenu();
    }
}
void thanksLine()
{
    cout<<"\n\n Thank You for Using My App...\n\n";
}

};
/*int main()
{
    DLinkedList d;
    d.mainmenu();
    cout<<"\n\n Thank You For Using My App...\n\n";
}*/
```

Combine LinkList Operations –

```
#include<iostream>
#include<conio.h>
#include"singlelist.cpp"
#include"doublylist.cpp"
class MainLinkList
{
public:
    SLinkList s;
    DLinkList d;
    void MainLinkListModule()
    {
        system("cls");
        cout<<"----- Welcome In Link List Project -----\\n\\n";
        cout<<"1. Singly Link List \\n\\n";
        cout<<"2. Doubly Link List \\n\\n";
        cout<<"3. Exit \\n\\n";
        cout<<"Choose any one of list....";
        char c = getch();
        if ( c == '1'){
            s.mainmenu();
        }
        else if ( c == '2'){
            d.mainmenu();
        }
        else if ( c == '3'){
            cout<<"\\n\\n Thank You for Using My App..\\n\\n";
            cout<<"Design & Coded By -- Pooja Khandelwal.. \\n\\n";
            exit(0);
        }

    }

};

int main()
{
    MainLinkList mll;
```

Twitter TimeLine Based on Link List

```
mll.MainLinkListModule();  
cout<<"\n\n Thank You for Using My App..\n\n";  
cout<<"Design & Coded By -- Pooja Khandelwal.. \n\n";  
}
```

Main Link List –

```
----- Welcome In Link List Project -----  
1. Singly Link List  
2. Doubly Link List  
3. Exit  
Choose any one of list....
```

Singly Link List –

INSERTION OPERATION

1. Insert At Beginning
2. Insert At End
3. Insert At Giving Position

do with one operation

SINGLY LINKLIST --

* Head -> [TWEET1001, ki, ji] -> NULL

Do you want to Continue .. Press 1 else press any key..|

DELETE OPERATIONS..

1. DELETE FROM BEGINNING
2. DELETE FROM END
3. DELETE FROM GIVEN POSITION
4. Go to Main Menu
5. Exit.

Search Tweet in Nodes --

* Head ->

Enter User Name To Find Tweets -- ki

[TWEET1001, ki, ji]

Counted Node in LinkList..

Total Nodes--1 Nodes Availabed

Doubly Link List -

INSERTION OPERATION

1. Insert At Beginning
2. Insert At End
3. Insert At Giving Position

DOUBLY LINKLIST --

1. FORWARD TRAVERSAL--
2. BACKWARD TRAVERSAL--
3. Go To Main Menu--

DELETE OPERATIONS..

1. DELETE FROM BEGINNING
2. DELETE FROM END
3. DELETE FROM GIVEN POSITION
4. Go to Main Menu
5. Exit.

Search Tweet in Nodes --

* Head ->

Enter User Name To Find Tweets -- krish

[TWEET1001, krish, hi jiya]

Counted Node in LinkList..

Total Nodes--1 Nodes Availabed

Github QR –



Behance QR –



Full Complexity Table

Operation	Singly Time	Doubly Time	Space	Notes
Insert at head	$O(1)$	$O(1)$	$O(1)$	Singly: 2 ops Doubly: 4 ops
Insert at tail	$O(n) / O(1)^*$	$O(1)$	$O(1)$	* $O(1)$ with tail pointer maintained
Insert at position k	$O(k)$	$O(k)$	$O(1)$	Traverse k steps; doubly needs 4 rewires
Delete head	$O(1)$	$O(1)$	$O(1)$	Move head forward
Delete tail	$O(n)$	$O(1) \star$	$O(1)$	$\star$ DOUBLY KEY ADVANTAGE: use tail.prev
Delete with pointer	$O(n)$	$O(1) \star$	$O(1)$	$\star$ Doubly: both neighbours known instantly
Delete by value	$O(n)$	$O(n)$	$O(1)$	Must search first; unlink is $O(1)$
Forward traversal	$O(n)$	$O(n)$	$O(1)$	Visit every node once
Backward traversal	N/A — impossible	$O(n)$	$O(1)$	Only doubly can go backwards
Search	$O(n)$	$O(n)$	$O(1)$	Binary search impossible — no random access
Count (counter var)	$O(1)$	$O(1)$	$O(1)$	Maintain size variable — always do this
Reverse	$O(n)$	$O(n)$	$O(1)$	Swap next/prev for all n nodes in-place

When to Use Which

Use Case	Best Choice	Reason
Twitter timeline, feed	Doubly	$O(1)$ delete anywhere, both-direction traversal
Browser back/forward	Doubly	Traverse both directions, $O(1)$ add/remove ends
Undo/Redo system	Doubly	Walk backward and forward through actions
LRU Cache	Doubly + HashMap	$O(1)$ delete any node when cache is full
Simple stack (LIFO)	Singly	Only need push/pop at head, less memory
Hash table chaining	Singly	Simple forward-only chain, less overhead
Print job queue (FIFO)	Singly + tail ptr	Enqueue at tail $O(1)$, dequeue at head $O(1)$
Playlist with prev/next	Doubly	Song navigation in both directions

Summary

Twitter use case: Doubly linked list. New tweets insert at head in $O(1)$. Any tweet deletes in $O(1)$.

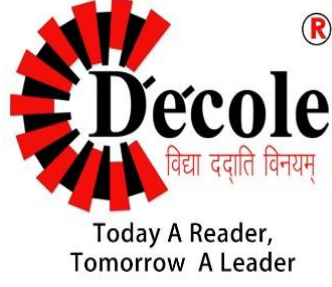
Timeline scrolls forward via next, backward via prev.

Singly operations: Insert head $O(1)$, insert tail $O(1)$, insert pos $O(k)$, delete head $O(1)$, delete tail $O(n)$, delete middle $O(n)$, traversal $O(n)$, search $O(n)$, count $O(1)$, reverse $O(n)$.

Doubly operations: Insert head $O(1)$, insert tail $O(1)$, insert pos $O(k)$, delete head $O(1)$, delete tail $O(1)$, delete middle $O(1)$ with ptr, forward $O(n)$, backward $O(n)$.

Marks doubly's advantages. The price: 8 extra bytes per node for the prev pointer.

The fundamental trade-off: Arrays give $O(1)$ random access but $O(n)$ insert/delete. Linked lists give $O(1)$ insert/delete at known positions but $O(n)$ access. Choose based on which operation is most frequent in your application.



THANK YOU.
